

REQUIREMENT SPECIFICATION & DESIGN DOCUMENT

Project: Student Management System (C++ Console Application) Course
Group: Group 06

1. Functional Requirements

The Student Management System provides comprehensive functionalities tailored for different user roles:

- **Authentication:** Secure Login, Logout, and Change Password features for all users.
- **Admin Management:** Full CRUD (Create, Read, Update, Delete) operations for Students, Teachers, Classes, and Subjects.
- **Teacher Operations:** View assigned classes, manage teaching schedules, and efficiently enter or modify student grades.
- **Student Operations:** Access personal profiles, view class schedules, register or drop subjects, and track academic results.

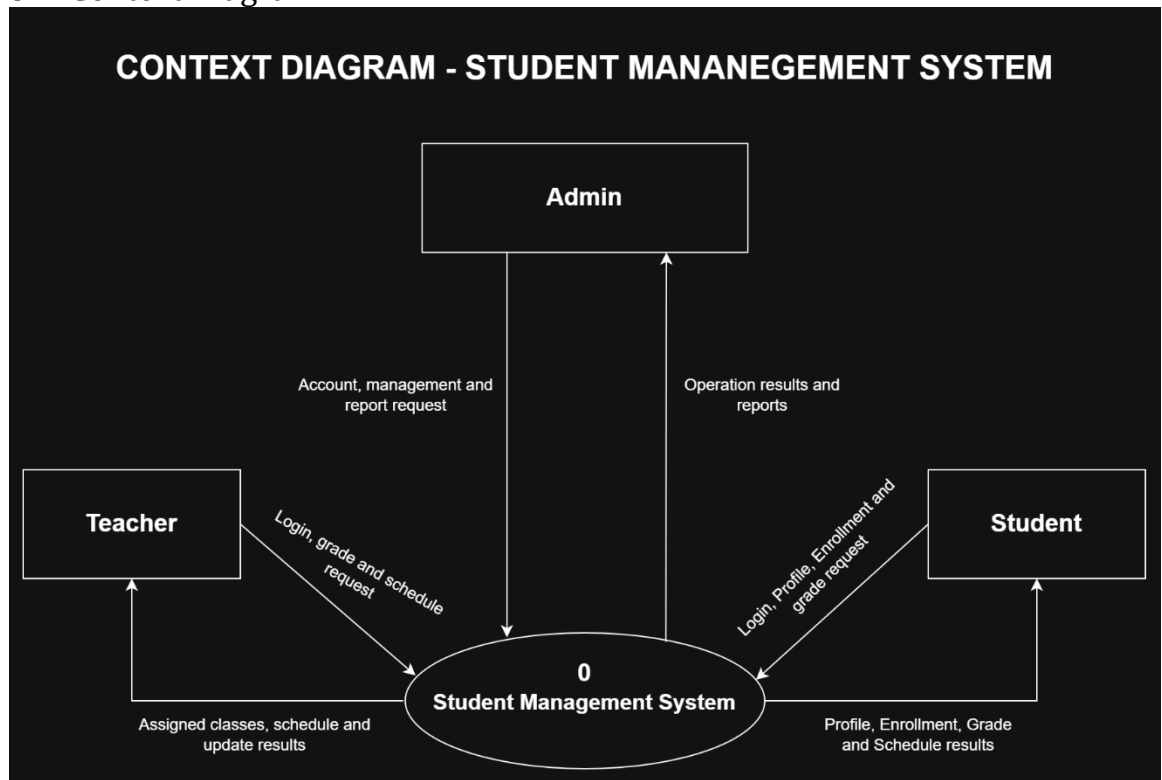
2. Non-Functional Requirements

- **Usability:** The text-based Console Interface is designed for quick navigation using simple numeric inputs (e.g., keys 1-9 for menu selection, 0 for returning or exiting).
- **Security:** System features are strictly isolated based on user authorization and role verification.
- **Technology Stack:** The application is developed using C++17, integrated with a relational database system (MySQL) for data persistence, and compiled via CMake.

3. Data Flow Diagram

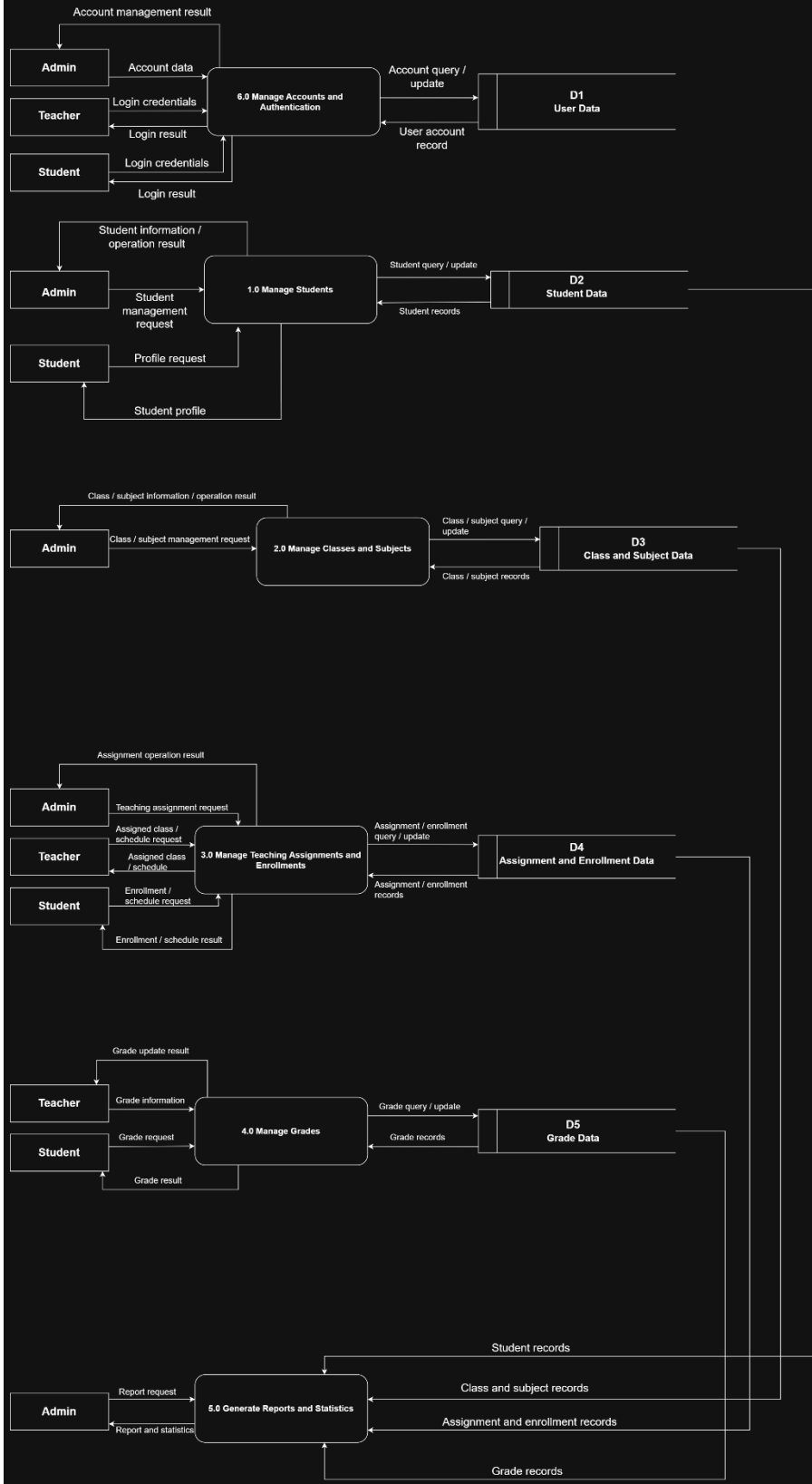
The Data Flow Diagrams (DFD) systematically map out how information is processed and transferred across the system.

- 3.1. Context Diagram:

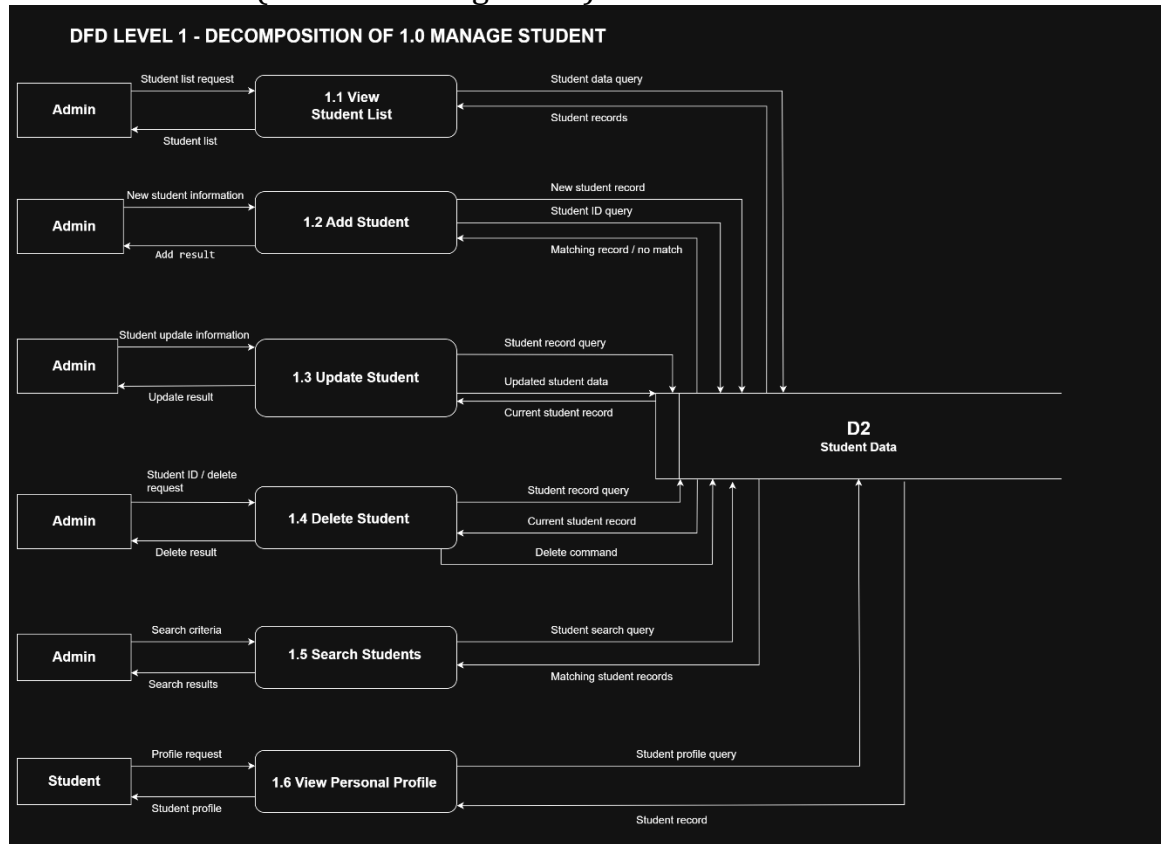


- 3.2. DFD Level 0:

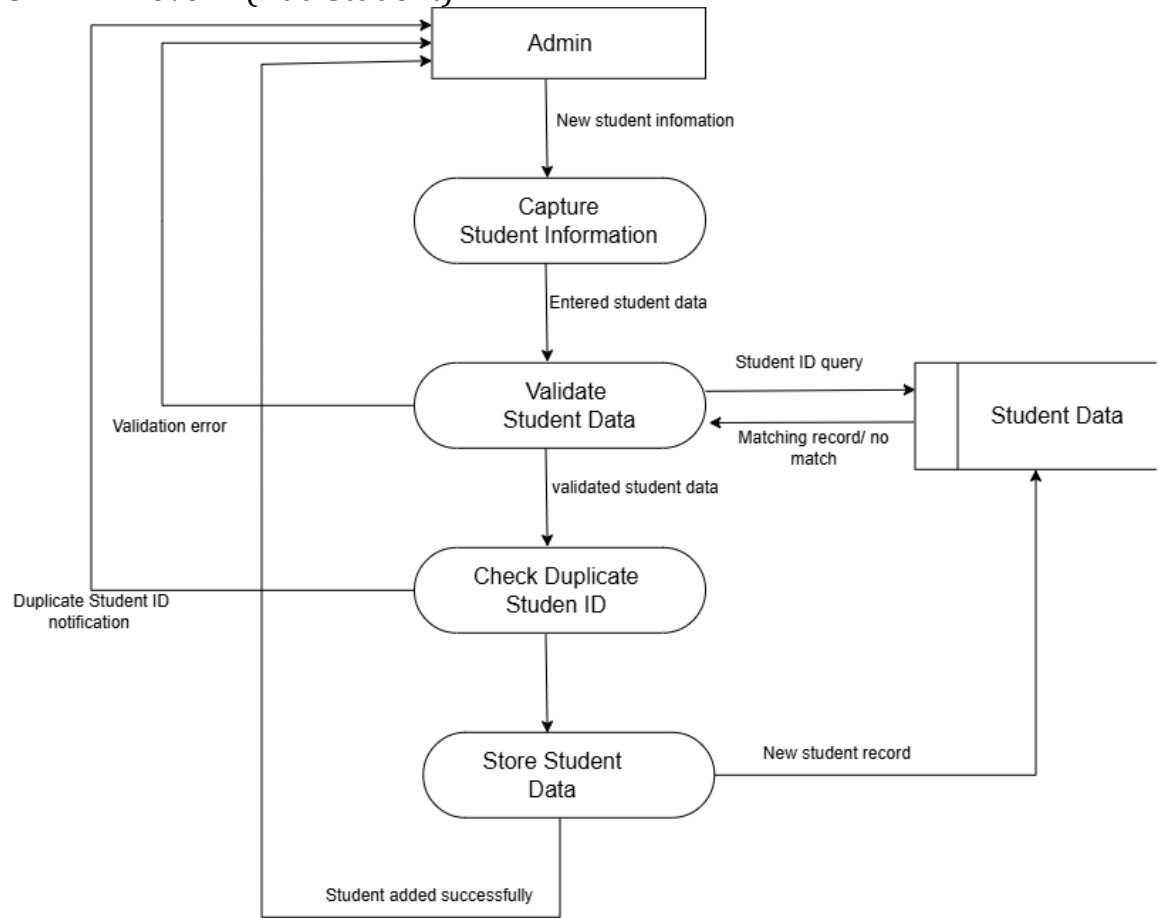
DFD LEVEL 0 - MAIN SYSTEM PROCESSES



- 3.3. DFD Level 1 (Student Management):

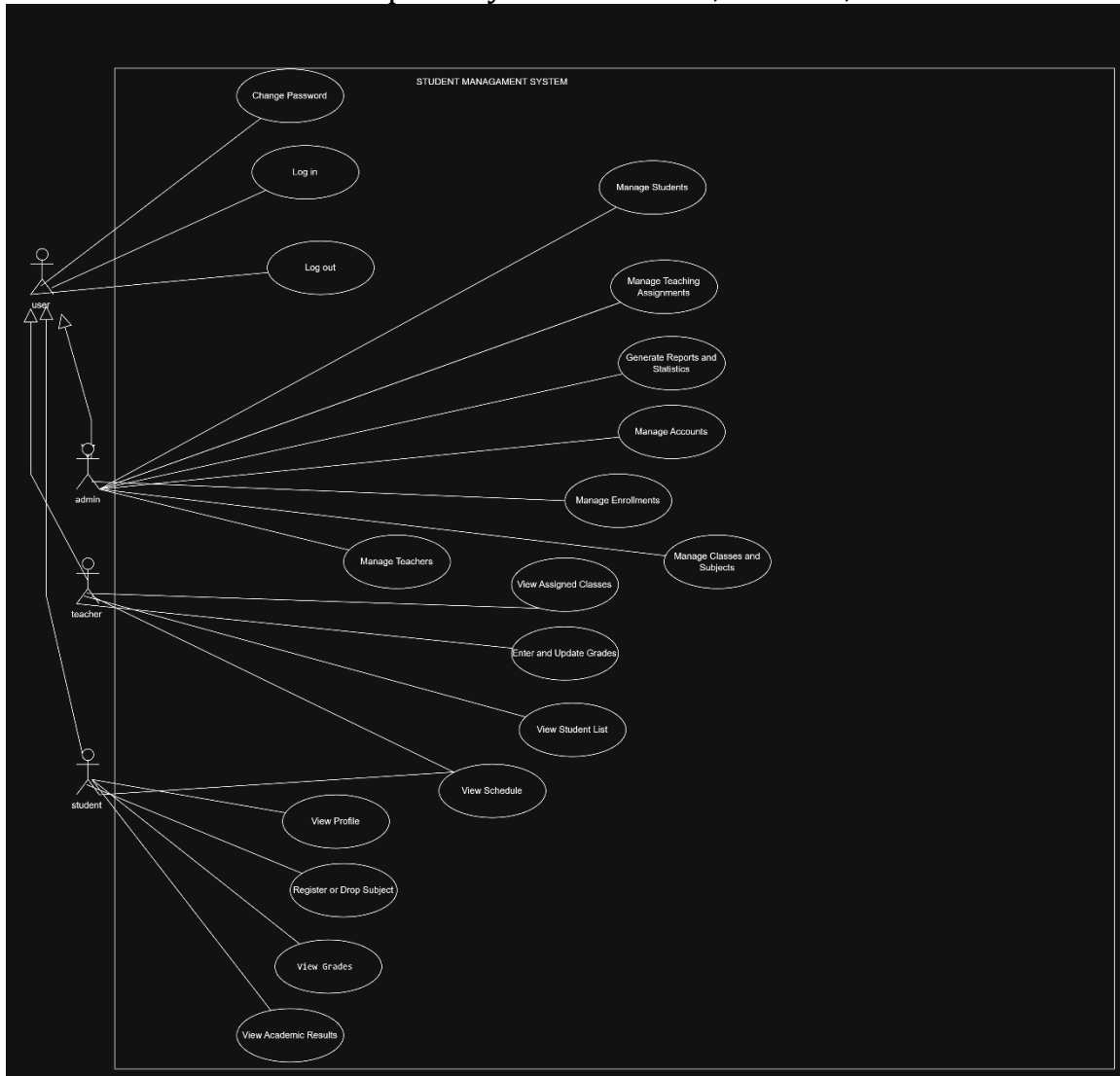


- 3.4 DFD Level 2 (Add Student):



4. Use Case Diagram

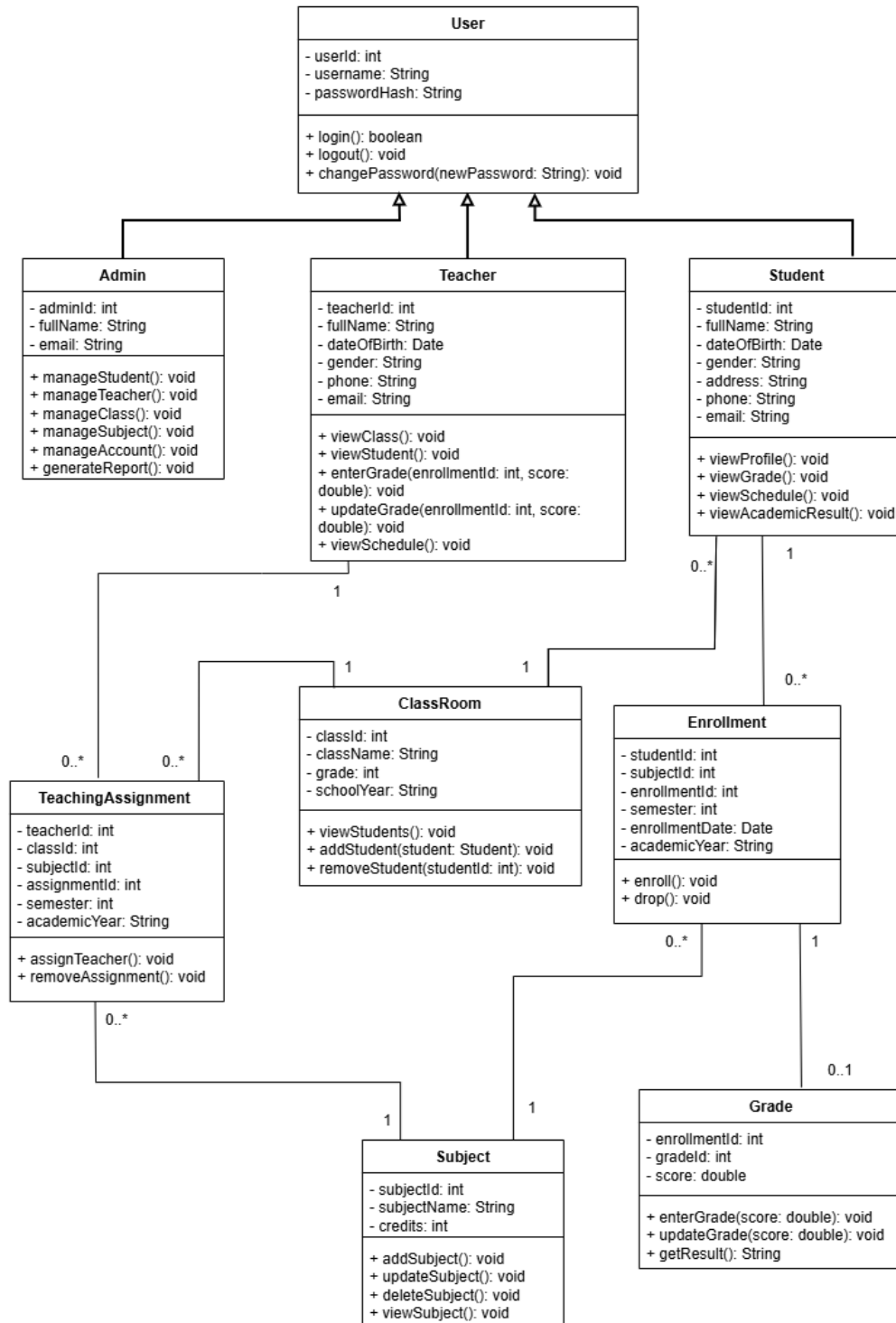
The Use Case Diagram visualizes the specific system interactions and accessible features for the primary actors: Admin, Teacher, and Student.



5. Class Diagram

The Class Diagram outlines the Object-Oriented Programming (OOP) architecture, detailing the classes, attributes, methods, and structural

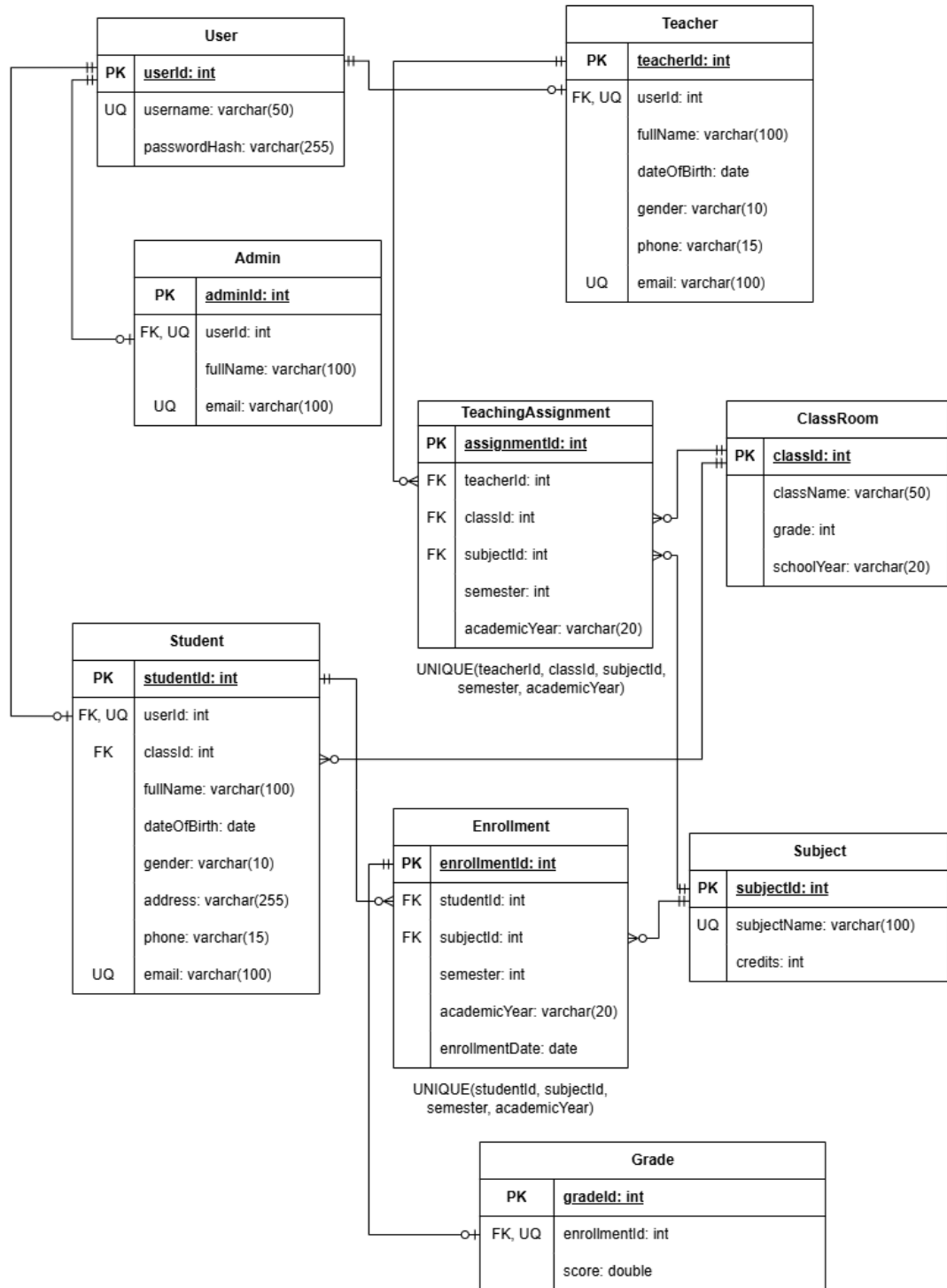
relationships implemented in the C++ source code.



6. Data Model

The database architecture guarantees data integrity and scalable storage for all academic records.

- 6.1. Entity Relationship Diagram (ERD):



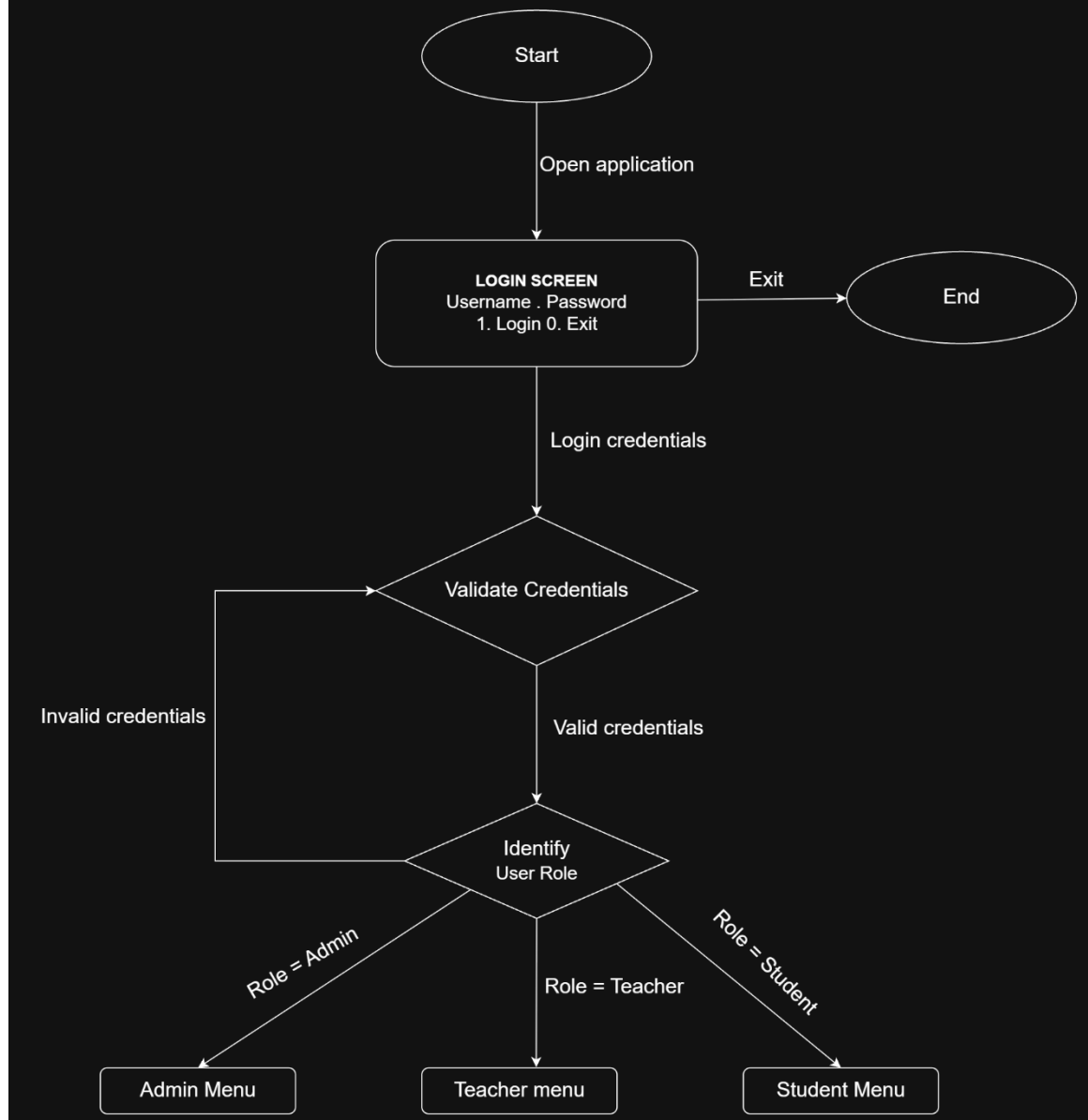
- 6.2. Database Schema: The foundational User entity serves as a base for role-specific attributes. Core operational tables, including Classroom, Subject, TeachingAssignment, Enrollment, and Grade, are robustly connected using Foreign Keys to manage the academic workflow seamlessly.

7. Interface Design Description

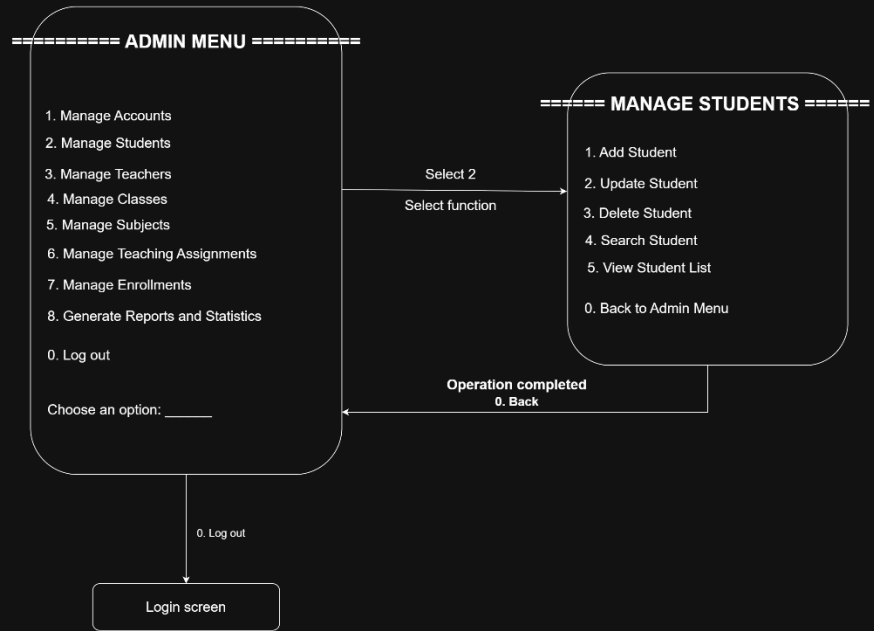
Given the C++ console environment, the system utilizes a streamlined, text-based navigation model.

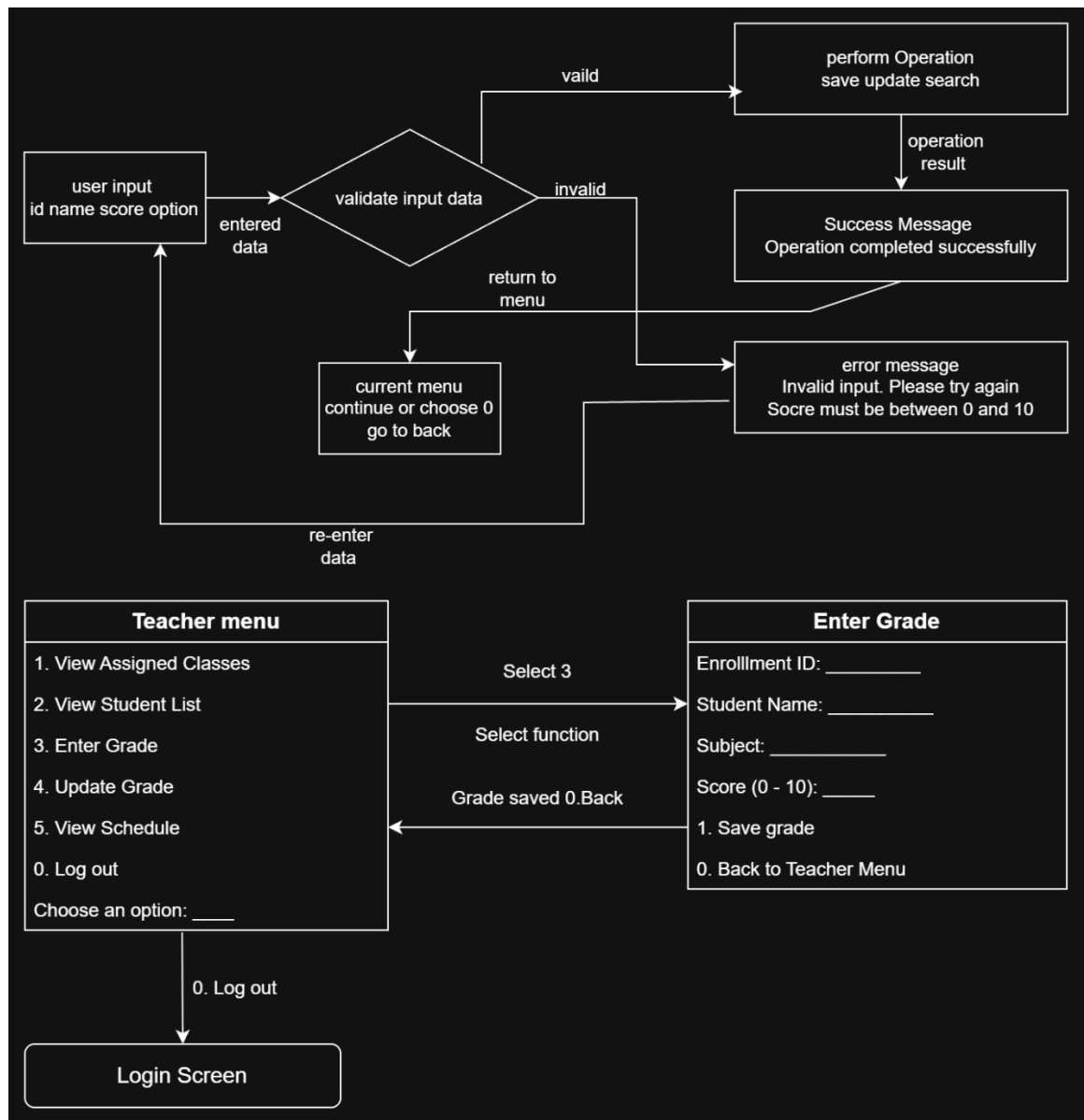
- 7.1. Console UI Mockups: All screens maintain consistent headers, clear prompt instructions, and numbered menu selections (Admin Menu, Teacher Menu, Student Menu).

LOGIN AND ROLE NAVIGATION



ADMIN MENU NAVIGATION





- 7.2. System Navigation & Validation: The architecture includes strict input validation loops to prevent application crashes, immediately returning actionable error messages or success confirmations upon

data entry.

